Group 105 - Laurier Lefebvre, Owen MacDonald, Johnny Weng Lin, Ayush Srivastava

Relational Schema

Users(emailAddress, password)

RegularUser(FOREIGN KEY (emailAddress) REFERENCES Users)

Admin(FOREIGN KEY (emailAddress) REFERENCES Users)

Genre(genreName)

favourite_genre(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (genreName) REFERENCES Genre)

Artist(artistId, name, ageLimit)

favourite_artist(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (artistId) REFERENCES Artist)

artist_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (artistId) REFERENCES Artist)

Venue(address, venueName, capacity)

venues_near_me(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (address) REFERENCES Venue)

venue_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (address) REFERENCES Venue)

concert(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, concertDate)

Tickets(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, FOREIGN KEY (concertDate) REFERENCES concert, section, seatNumber, price)

online_purchase(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES RegularUser)

in-person_purchase(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

scan(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

Application program

Main Menu (ConcertPro.java):
This is the menu where users can select what option they want to do. Selecting a correct option brings them to the menu for that option, selecting 6 for “Quit” closes the connection and ends the program, and selecting a wrong option prompts the user for a number between 1 and 6.

Example of choosing an option not between 1 and 6:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 7
Invalid option. Please choose 1 to 6.
```

Example of quitting:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 6
Goodbye.
cs421g105@winter2026-comp421:~/java$
```

Task 1: Buy Ticket

This option asks for the user's login information, then if login is correct asks for the ticket information, asking for age verification if the artist has an age restriction, then if the ticket exists and is not already purchased, purchases the ticket for the user (adds an online_purchase relation).

Example of entering bad login information:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 1

=== Buy Ticket ===
Enter email address: notanemail.com
Enter password: uhmidk
User info is incorrect.
```

Example of age verification failing (artistID=1 has an ageLimit of 18):

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 1

=== Buy Ticket ===
Enter email address: john.doe22@anonymous.ca
Enter password: password123
Enter artist ID: 1
Enter venue address: 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
Enter concert date (YYYY-MM-DD): 2026-03-22
Enter section: 202
Enter seat number: 13
Enter your age: 16
Age verification failed. Purchase denied.
```

Example of age verification successful (again, artistID=1 has an ageLimit=18):

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 1

=== Buy Ticket ===
Enter email address: john.doe22@anonymous.ca
Enter password: password123
Enter artist ID: 1
Enter venue address: 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
Enter concert date (YYYY-MM-DD): 2026-03-22
Enter section: 202
Enter seat number: 13
Enter your age: 20
Purchase successful.
```

Example of a non-existent ticket (age verification doesn't happen because artistId=67 has ageLimit=null):

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 1

=== Buy Ticket ===
Enter email address: john.doe22@anonymous.ca
Enter password: password123
Enter artist ID: 67
Enter venue address: not a correct address
Enter concert date (YYYY-MM-DD): 2025-01-23
Enter section: 2
Enter seat number: 1
That ticket does not exist.
```

Example of an already purchased ticket (we bought it in a previous example):

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 1

=== Buy Ticket ===
Enter email address: john.doe22@anonymous.ca
Enter password: password123
Enter artist ID: 1
Enter venue address: 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
Enter concert date (YYYY-MM-DD): 2026-03-22
Enter section: 202
Enter seat number: 13
That ticket has already been sold.
```

Task 2: Admin Ticket Purchase

This option asks for the admin's login information, then if login is correct asks for the ticket information, then if the ticket exists and is not already purchased, makes the in_person_purchase relation to signify that the specific ticket has been sold to someone in-person by that admin. There is no age verification since the admin would do that in-person.

Example of entering bad login information:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 2

=== Admin Ticket Kiosk ===
Enter admin email address: notanadmin.com
Enter password: wow
Admin login failed.
```

Example of a non-existent ticket:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 2

=== Admin Ticket Kiosk ===
Enter admin email address: gandalf@middle.earth.com
Enter password: YOUSHALLNOTPASS
Enter artist ID: 76
Enter venue address: my house
Enter concert date (YYYY-MM-DD): 2023-03-30
Enter section: 2
Enter seat number: 2
That ticket does not exist.
```

Example of a successfully purchased ticket:

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 2

=== Admin Ticket Kiosk ===
Enter admin email address: gandalf@middle.earth.com
Enter password: YOUSHALLNOTPASS
Enter artist ID: 480
Enter venue address: 164 Sunnyside Road, Ottawa, Canada
Enter concert date (YYYY-MM-DD): 2026-03-27
Enter section: 423
Enter seat number: 25
Purchase successful.
```

Example of an already purchased ticket (the ticket we purchased in the above example):

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 2

=== Admin Ticket Kiosk ===
Enter admin email address: gandalf@middle.earth.com
Enter password: YOUSHALLNOTPASS
Enter artist ID: 480
Enter venue address: 164 Sunnyside Road, Ottawa, Canada
Enter concert date (YYYY-MM-DD): 2026-03-27
Enter section: 423
Enter seat number: 25
That ticket has already been sold.
```

Task 3: Add favourite artist

This task allows a user to add an artist to their list of favourite artists,. The program prompts the user for a regular user email address and an artist ID which then executes an Insert statement into the favourite artists relation. This matches the application design where a feature takes user information and artist information and creates the corresponding tuple.

This screenshot shows a possible outcome of this task. In this example, the program attempts to insert a favourite artist that already exists for that user. This caused DB2 to return a duplicate error which indicates that the tuple already exists. This application handles it properly by returning the error statement and allowing the user to continue with the application.

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 3

Add Favourite Artist
Enter user email address: harry.potter@hogwarts.com
Enter artist ID: 123
SQL error:
Code: -803
State: 23505
DB2 SQL Error: SQLCODE=-803, SQLSTATE=23505, SQLERRMC=1;CS421G105.FAVOURITE_ARTIST, DRIVER=4.33.32
```

In the following example, the program does the insert in the similar format but this time with a tuple that doesn't exist currently in the database. It can be seen that the insertion succeeds and the application prints the confirmation message. Then the application returns to the main menu.

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 3

Add Favourite Artist
Enter user email address: john.doe22@anonymous.ca
Enter artist ID: 123
Favourite artist added successfully.

==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option:
```

Task 4: Show recommended concerts

This task first asks for a user's email address. Once the email address is entered, the task will either display:

A message indicating the entered email address is invalid. In this case, the email address does not exist in the database.

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 4

Show recommended concerts for a specific user
depending on the user's favourite artists and genres.

Enter user email address:
invalid.email@address.ca

No user found with this email address.
```

A message indicating the entered email address is valid, but it doesn't have any concert recommendations. In this case, a user does exist with this email address, however, either they do not have any favourite artists and/or genres or they do but no artists associated with them are performing in a concert.

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 4

Show recommended concerts for a specific user
depending on the user's favourite artists and genres.

Enter user email address:
oceanwaves67@amazon.ca

SQL Query:
SELECT DISTINCT A.name, C.address, C.concertDate FROM concert C, Artist A WHERE C.artistId = A.artistId AND (C.art
istId IN (SELECT artistId FROM favourite_artist WHERE emailAddress = 'oceanwaves67@amazon.ca') OR C.artistId IN (S
ELECT G.artistId FROM artist_genre G, favourite_genre F WHERE G.genreName = F.genreName AND F.emailAddress = 'ocea
nwaves67@amazon.ca')) ORDER BY C.concertDate, A.name

List of concerts:

User exists but has no recommended concerts.
```

A list of upcoming concerts that are either performed by the user's favourite artists or artists categorized by a genre that's in the user's favourite genre. The list is ordered ascending by

date, this way the concerts that are happening sooner are displayed first.

```
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Feature Five
6. Quit
Select an option: 4

Show recommended concerts for a specific user
depending on the user's favourite artists and genres.

Enter user email address:
frodo.baggins@hobbit.ca

SQL Query:
SELECT DISTINCT A.name, C.address, C.concertDate FROM concert C, Artist A WHERE C.artistId = A.artistId AND (C.art
istId IN (SELECT artistId FROM favourite_artist WHERE emailAddress = 'frodo.baggins@hobbit.ca') OR C.artistId IN (
SELECT G.artistId FROM artist_genre G, favourite_genre F WHERE G.genreName = F.genreName AND F.emailAddress = 'fro
do.baggins@hobbit.ca')) ORDER BY C.concertDate, A.name

List of concerts:

Artist: Avicii
Venue: 164 Sunnyside Road, Ottawa, Canada
Date: 2026-03-27

Artist: Dire Straits
Venue: 456 South Mountain Boulevard, Vancouver, Canada
Date: 2026-04-13

Artist: Fleetwood Mac
Venue: 456 South Mountain Boulevard, Vancouver, Canada
Date: 2026-05-12

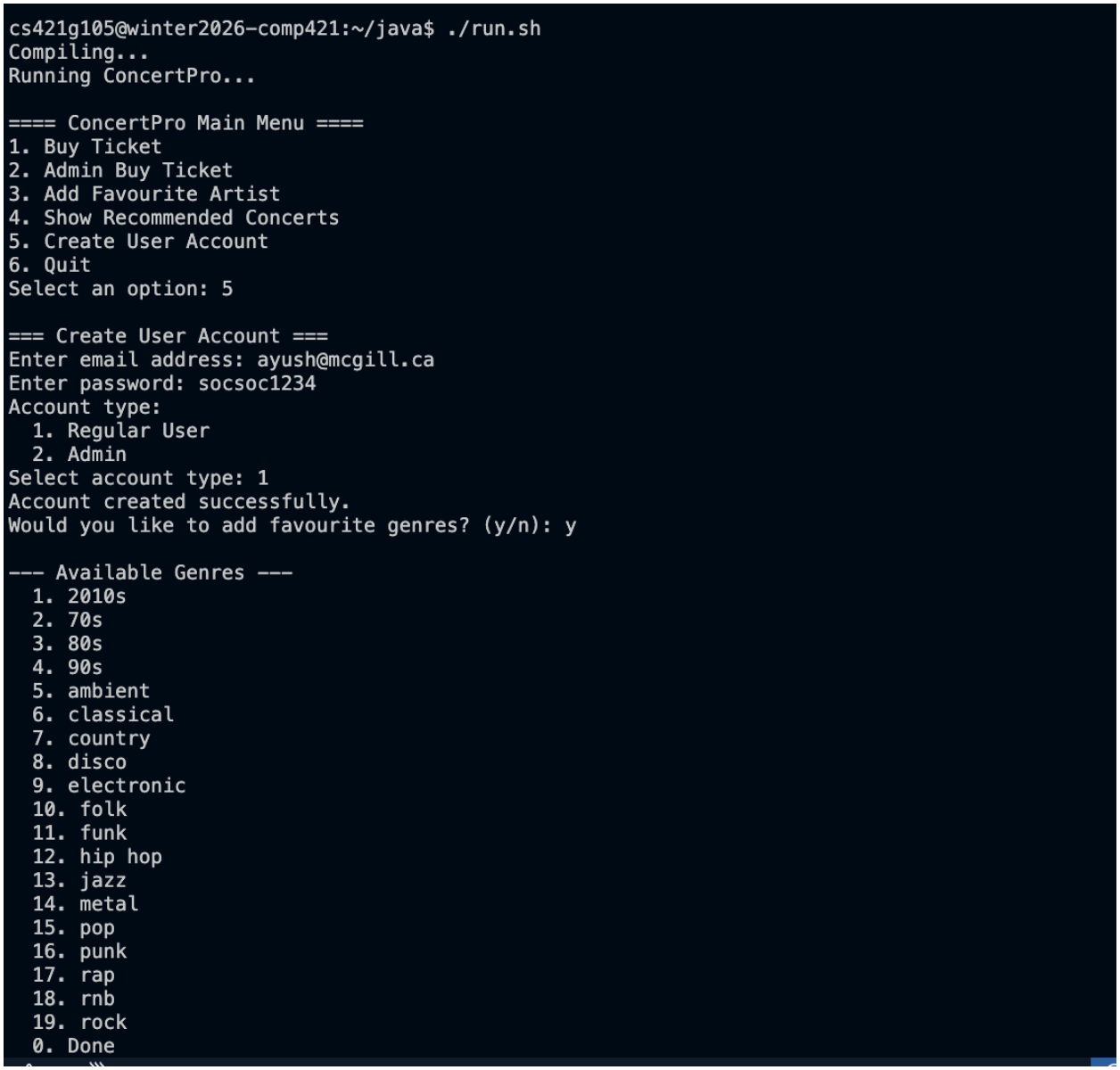
Artist: Avicii
Venue: 164 Sunnyside Road, Ottawa, Canada
Date: 2027-12-12

DONE.
```

Task 5: Create User Accounts

This feature allows a new user to register an account in the ConcertPro system. The application prompts for an email address, password, and account type (Regular User or Admin). It first checks whether the email already exists in the database to prevent duplicates. If the email is available, it inserts a record into the Users table and the appropriate subtype table (RegularUser or Admin) within a single transaction, rolling back both inserts if either fails.

After account creation, the user is presented with a sub-menu generated from a database query listing all available genres from the Genre table. The user can select genres by number to add them as favourites, which are inserted into the favourite_genre table. The sub-menu loops until the user selects "Done," allowing multiple genres to be added in one session. Duplicate favourite entries are detected and skipped with a message.


cs421g105@winter2026-comp421:~/java$ ./run.sh
Compiling...
Running ConcertPro...
==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Create User Account
6. Quit
Select an option: 5
=== Create User Account ===
Enter email address: ayush@mcgill.ca
Enter password: socsoc1234
Account type:
1. Regular User
2. Admin
Select account type: 1
Account created successfully.
Would you like to add favourite genres? (y/n): y
--- Available Genres ---
1. 2010s
2. 70s
3. 80s
4. 90s
5. ambient
6. classical
7. country
8. disco
9. electronic
10. folk
11. funk
12. hip hop
13. jazz
14. metal
15. pop
16. punk
17. rap
18. rnb
19. rock
0. Done

```
  0. Done
Select a genre to add: 8
Added 'disco' to your favourites.

--- Available Genres ---
  1. 2010s
  2. 70s
  3. 80s
  4. 90s
  5. ambient
  6. classical
  7. country
  8. disco
  9. electronic
  10. folk
  11. funk
  12. hip hop
  13. jazz
  14. metal
  15. pop
  16. punk
  17. rap
  18. rnb
  19. rock
  0. Done
Select a genre to add: 0

==== ConcertPro Main Menu ====
1. Buy Ticket
2. Admin Buy Ticket
3. Add Favourite Artist
4. Show Recommended Concerts
5. Create User Account
6. Quit
Select an option:
```

Stored Procedure:

(a): The stored procedure ApplyGenreDiscount takes a genre name and a discount percentage as input and applies that discount to all unsold tickets for concerts performed by artists of that genre. It uses a cursor to iterate through each qualifying ticket, computes the new discounted price, and updates the ticket's price in the Tickets table. A ticket is considered unsold if it does not appear in either the online_purchase or in_person_purchase tables. The procedure returns the total number of tickets that were discounted via an output parameter.

(b):

```
CONNECT TO COMP421@


CREATE OR REPLACE PROCEDURE ApplyGenreDiscount(
   IN p_genreName VARCHAR(52),
   IN p_discountPct DECIMAL(5,2),
   OUT p_totalDiscounted INTEGER
)
```

```
LANGUAGE SQL
BEGIN
   DECLARE v_artistId INTEGER;
   DECLARE v_address VARCHAR(255);
   DECLARE v_concertDate DATE;
   DECLARE v_section VARCHAR(20);
   DECLARE v_seatNumber INTEGER;
   DECLARE v_oldPrice DECIMAL(10,2);
   DECLARE v_newPrice DECIMAL(10,2);
   DECLARE v_count INTEGER DEFAULT 0;
   DECLARE v_done INTEGER DEFAULT 0;

   -- Cursor: unsold tickets for concerts by artists of the given genre
   DECLARE ticket_cursor CURSOR FOR
       SELECT t.artistId, t.address, t.concertDate, t.section, t.seatNumber, t.price
       FROM Tickets t
       JOIN concert c ON t.artistId = c.artistId
                     AND t.address = c.address
                     AND t.concertDate = c.concertDate
       JOIN artist_genre ag ON c.artistId = ag.artistId
       WHERE ag.genreName = p_genreName
         AND NOT EXISTS (
             SELECT 1 FROM online_purchase op
             WHERE op.artistId = t.artistId AND op.address = t.address
               AND op.concertDate = t.concertDate AND op.section = t.section
               AND op.seatNumber = t.seatNumber
         )
         AND NOT EXISTS (
             SELECT 1 FROM in_person_purchase ip
             WHERE ip.artistId = t.artistId AND ip.address = t.address
               AND ip.concertDate = t.concertDate AND ip.section = t.section
               AND ip.seatNumber = t.seatNumber
         );

   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

   OPEN ticket_cursor;

   fetch_loop: LOOP
       FETCH ticket_cursor INTO v_artistId, v_address, v_concertDate, v_section,
v_seatNumber, v_oldPrice;
```

```
      IF v_done = 1 THEN
          LEAVE fetch_loop;
      END IF;

      SET v_newPrice = DECIMAL(v_oldPrice * (100 - p_discountPct) / 100, 10, 2);

      UPDATE Tickets
      SET price = v_newPrice
      WHERE artistId = v_artistId
        AND address = v_address
        AND concertDate = v_concertDate
        AND section = v_section
        AND seatNumber = v_seatNumber;

      SET v_count = v_count + 1;
   END LOOP fetch_loop;

   CLOSE ticket_cursor;

   SET p_totalDiscounted = v_count;
END @
```

(c) and (d)

The stored procedure was created using db2 -td@ -f stored_procedure.sql and then called with genre 'pop' and a 15% discount. The before and after SELECT queries show that the procedure correctly identified and discounted 10 unsold tickets. For example, the ticket in section 211 seat 20 went from $149.93 to $127.44, and section 101 seat 4 went from $222.19 to $188.86 — both exactly 85% of the original price. The first three tickets (section 202 seat 13, section 310 seat 30, General Admission seat 12) remained unchanged because they had already been purchased and thus were excluded by the procedure's NOT EXISTS checks against the online_purchase and in_person_purchase tables.

```
cs421g105@winter2026-comp421:~$ db2 "SELECT t.artistId, t.section, t.seatNumber, t.price FROM Tickets t JOIN artist_genre ag ON t.artistId = ag.artistId
 WHERE ag.genreName = 'pop' FETCH FIRST 10 ROWS ONLY"
db2 "CALL ApplyGenreDiscount('pop', 15.00, ?)"
db2 "SELECT t.artistId, t.section, t.seatNumber, t.price FROM Tickets t JOIN artist_genre ag ON t.artistId = ag.artistId WHERE ag.genreName = 'pop' FETC
H FIRST 10 ROWS ONLY"

ARTISTID    SECTION              SEATNUMBER  PRICE
----------- -------------------- ----------- ------------
      10001 202                           13       110.53
      10001 310                           30        80.90
      10001 General Admission             12       235.45
      10001 211                           20       149.93
      10001 219                           19       144.35
      10001 413                           16        82.59
      10001 327                           10        90.09
      10001 224                            1       139.20
      10001 208                           14       127.99
      10001 101                            4       222.19

  10 record(s) selected.


  Value of output parameters
  --------------------------
  Parameter Name  : P_TOTALDISCOUNTED
  Parameter Value : 10

  Return Status = 0

ARTISTID    SECTION              SEATNUMBER  PRICE
----------- -------------------- ----------- ------------
      10001 202                           13       110.53
      10001 310                           30        80.90
      10001 General Admission             12       235.45
      10001 211                           20       127.44
      10001 219                           19       122.69
      10001 413                           16        70.20
      10001 327                           10        76.57
      10001 224                            1       118.32
      10001 208                           14       108.79
      10001 101                            4       188.86

  10 record(s) selected.

cs421g105@winter2026-comp421:~$
```

Indexes and Creativity:

For the indexing requirement, we created a useful index on Venue(venueName). We chose this attribute because venueName is not a primary key in our schema, while address is the identifying attribute for Venue. This makes venueName an appropriate candidate for indexing under the project instructions, which specify that indexes on primary keys and unique constraints should not be counted. We selected this index because our application and our earlier SQL work frequently identify venues by name before retrieving related information, so lookups by venue name are expected to benefit from faster access.

```
db2 => CREATE INDEX idx_venue_name ON Venue(venueName);
DB20000I  The SQL command completed successfully.
```

This index is expected to improve queries and modifications that first search for a venue by its name. In our project work, this is relevant for venue-based lookups and for administrative operations that identify a venue such as “Centre Bell” before working with related concerts or tickets. Without the index, DB2 may need to scan the full Venue table to find the matching row. With the index, the database can locate the venue more efficiently.

For the creativity component, we extended our indexing work by creating a second, more sophisticated index on favourite_genre(genreName, emailAddress). We had already created the index on Venue(venueName) for the main indexing requirement, so for the creativity part we chose a clustered composite index. This satisfies the requirement that one of the two indexes be more advanced, for example by indexing multiple attributes rather than only one. Because

this index is both composite and clustered, it is a stronger indexing choice than a basic single-column index.

```
db2 => CREATE INDEX idx_favourite_genre_genre_email ON favourite_genre(genreName, emailAddress) CLUSTER;
DB20000I  The SQL command completed successfully.
db2 =>
```

We chose this clustered composite index because it is directly relevant to the recommendation-style queries in our application. Our database uses favourite genres to connect user preferences to artists and then to concerts. Since these queries rely on matching rows by genreName, indexing genreName together with emailAddress supports that access pattern well. In a composite clustered index, the clustering follows the full key order, so rows are organized first by genreName and then by emailAddress within each genre. This means that rows with the same genre are stored more closely together, which is expected to improve the efficiency of genre-based lookups compared with scanning the whole favourite_genre table.

Visualization

The goal of this chart is to visualize the top 10 artists by number of tickets sold. This information could be useful to create a “trending artists” category. It would’ve been useful to have added an attribute “dateSold” to tickets sold in “online_purchase” and “in_person_purchase”. This would’ve allowed the query retrieving the data to only look at recent ticket sales, meaning the sales data is more relevant and references current “trending artists” instead of all time “trending artists”.

Here is the SQL used to retrieve the data required to make the chart:

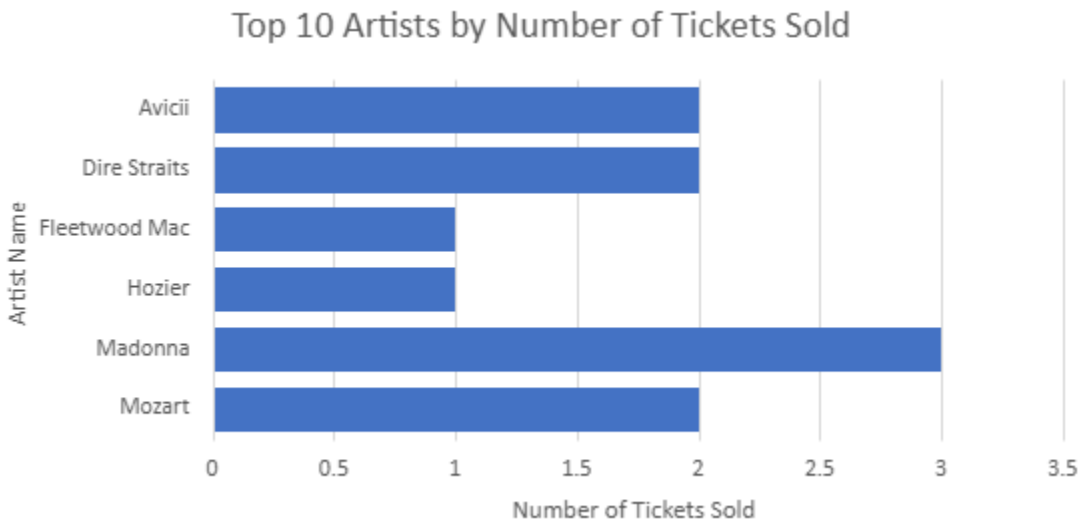
```
EXPORT TO result.csv OF DEL MODIFIED BY NOCHARDEL
SELECT A.name, COUNT(*) AS tickets_sold
FROM (SELECT artistId FROM online_purchase UNION ALL SELECT artistId FROM
in_person_purchase) P, Artist A
WHERE P.artistId = A.artistId
GROUP BY A.name
ORDER BY tickets_sold DESC
FETCH FIRST 10 ROWS ONLY
```

```
db2 => cs421g105@winter2026-comp421:~$ db2 "EXPORT TO result.csv OF DEL MODIFIED BY NOCHARDEL SELECT A.name, COUNT
(*) AS tickets_sold FROM (SELECT artistId FROM online_purchase UNION ALL SELECT artistId FROM in_person_purchase)
P, Artist A WHERE P.artistId = A.artistId GROUP BY A.name ORDER BY tickets_sold DESC FETCH FIRST 10 ROWS ONLY"
SQL3104N  The Export utility is beginning to export data to file "result.csv".

SQL3105N  The Export utility has finished exporting "6" rows.


Number of rows exported: 6
```

Here is an image of the chart generated using this data:



This chart lists the top artists on the vertical axis and number of tickets sold on the horizontal axis. The chart only has 6 artists because there are only 6 artists that have sold tickets in the database. Obviously, if there were 10 artists or more in the database, only the top 10 selling artists would appear on the chart. Also, the number of tickets sold is quite low for each artist because of the low amount of data. Again, in a real world scenario, each artist would have way more ticket sales.

Contributions

Owen worked on task 4 (show recommended concerts) of the application program (Q4), data visualization (Q6) and helped formalize the report. Laurier made ConcertPro.java, the main menu for the program (Q4) that then calls the other java classes for their functionality. He also wrote features 1 and 2 (buy ticket and admin buy ticket) for Q4 and helped formalize the report. Johnny worked on the java task 3 where we added favourite artists of users as well as the indexes component and creativity component. Ayush wrote task 5 (create user account) of the application program (Q4) and worked on the stored procedure (Q3).